

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – INDUSTRY PROBLEM SOLVING TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO4 – Precision (BL4)	Assessment:	Industry Problem solving task
Weightage:	30%	Total Marks:	100
CO4 Statement:	Implement hierarchical memory systems by differentiating cache levels (L1, L2, L3), virtual memory with paging, and advanced memory technologies (DRAM, SRAM, NAND Flash, MRAM, RRAM) based on latency, bandwidth, and throughput characteristics.		
Student Name:	RAGAVI S	Reg. No.:	192565027
Section / Batch:	SLOT A (2025-29)	Date:	19-08-2026

Instructions to Students

- Identify the relevant concepts of cache hierarchy, SRAM, DRAM, MRAM, NAND Flash, RRAM, memory latency, bandwidth, and virtual memory paging.
- Analyze the memory-access requirements of smartphones, cloud servers, autonomous vehicles, edge AI devices, and gaming consoles, considering latency, bandwidth, capacity, and power consumption.
- Apply suitable memory-hierarchy techniques such as cache optimization, appropriate memory technology selection, data locality, paging optimization, and hierarchical memory organization.
- Compute performance measures such as memory access time, latency, bandwidth, throughput, speedup, hit/miss rates, and resource or power utilization where applicable.
- Interpret the results by evaluating performance improvements, memory bottlenecks, technology trade-offs, and the suitability of the proposed memory hierarchy for each application.

QUESTIONS

1. A large-scale e-commerce platform experiences slow response times during peak shopping periods when thousands of users simultaneously access product catalogs, customer records, and transaction data. Analyze the roles of cache memory, DRAM, and secondary storage in handling these workloads. Propose a memory hierarchy that reduces access latency, improves data locality, and supports high concurrent transaction throughput.
2. A robotic manufacturing system must process camera images, sensor readings, and control instructions with strict real-time deadlines. Evaluate the suitability of SRAM, DRAM, and non-volatile memory for storing frequently accessed control data, sensor information, and program code. Design a memory hierarchy that minimizes latency and ensures predictable memory access for real-time operations.
3. A video-editing workstation experiences delays while processing multiple streams of high-resolution 4K and 8K video. Analyze how cache size, DRAM capacity, memory bandwidth, and storage performance influence video-processing workloads. Recommend a hierarchical memory organization that improves data transfer rates, reduces processing delays, and supports smooth real-time editing.
4. A weather forecasting system performs complex numerical calculations on massive datasets containing atmospheric temperature, pressure, wind, and humidity measurements. Analyze how cache hierarchy, DRAM bandwidth, and large-capacity memory influence the performance of repeated numerical computations. Propose memory-hierarchy enhancements that improve data locality, reduce memory-access bottlenecks, and increase computational throughput.
5. A smart home gateway manages data from cameras, motion sensors, environmental sensors, and connected appliances while operating continuously with limited energy resources. Compare SRAM, DRAM, Flash, and other suitable memory technologies based on latency, capacity, persistence, and power consumption. Design an efficient memory hierarchy that enables rapid access to frequently used data while minimizing energy consumption and maintaining reliable long-term data storage.

Code of Conduct

I RAGAVI S / 192565027 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: RAGAVI S

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Q. No	Understanding of Industry Problem (20)	Application of Engineering Concepts (25)	Solution Design (25)	Use of Tools (15)	Analysis (10)	Professionalism(5)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 25	/ 25	/ 15	/ 10	/ 5	/ 100

Course Coordinator

Faculty Incharge

Question 1 — E-Commerce Platform Under Peak Load

Scenario: Thousands of concurrent users hit product catalogs, customer records, and transaction data during peak sales, causing slow response times.

1. Analysis of the Workload

- Product catalog reads: extremely read-heavy, highly repetitive (same best-sellers viewed by many users) — strong temporal and spatial locality, ideal for caching.
- Customer records: read-heavy with occasional writes (profile/address updates) — moderate locality, sensitive to consistency.
- Transaction data: write-heavy, must be durable and ACID-compliant — cannot rely on volatile cache alone.
- Peak concurrency multiplies working-set size, so a hierarchy sized only for average load will thrash.

2. Role of Each Memory Level

Memory Level	Role in this workload	Latency / Bandwidth
CPU L1/L2/L3 cache	Holds hot instruction paths of the request-handling code and small hot data (session counters, index nodes)	L1 ~1 ns, L2 ~3–5 ns, L3 ~15–30 ns
DRAM (server RAM)	Holds the in-memory working set: DB buffer pool, session state, JVM/app heap	~80–120 ns, tens of GB/s
Distributed in-memory cache (Redis/Memcached)	Acts as an application-level 'L4' — caches rendered catalog pages, price/stock lookups, avoiding DB hits entirely	~0.3–1 ms over network, but avoids disk I/O
NVMe SSD (secondary storage)	Persists transaction and order data, DB write-ahead logs, indexes too large for RAM	~20–100 μ s, 2–7 GB/s
HDD / object storage (cold tier)	Archived orders, logs, backups — rarely accessed	ms-level, high capacity, low cost/GB

3. Proposed Memory Hierarchy

A layered design, from fastest/smallest to slowest/largest:

- L1/L2/L3 CPU caches — unchanged hardware, but application code is written to be cache-friendly (compact data structures, avoiding pointer-chasing) so hot loops (price lookup, cart totals) stay in cache.
- DRAM buffer pool — size the database buffer pool to hold the 'hot' catalog and index pages (80/20 rule: ~20% of catalog generates ~80% of reads).
- Distributed cache tier (Redis) — read-through/write-back cache in front of the database for catalog pages and inventory counters; absorbs the bulk of read concurrency so it never reaches disk.
- NVMe-backed database storage with write-ahead logging — guarantees durability for transactions while keeping write latency low.
- Sharding/partitioning of the transaction database by customer or region — improves data locality and lets each shard's working set fit in its own DRAM+cache tier.
- Cold storage tier for historical orders — moved out of the hot path entirely, freeing DRAM/cache capacity for active traffic.

4. Performance Calculation (Average Memory Access Time)

Assume a request path with the following hit rates and access times per level:

L1 hit rate = 95%, time = 1 ns

L2 hit rate = 90% of L1 misses, time = 5 ns

L3 hit rate = 80% of L2 misses, time = 20 ns

DRAM access time = 100 ns

$$\text{AMAT} = T_{\text{L1}} + \text{MissRate}_{\text{L1}} \times (T_{\text{L2}} + \text{MissRate}_{\text{L2}} \times (T_{\text{L3}} + \text{MissRate}_{\text{L3}} \times T_{\text{DRAM}}))$$
$$\text{AMAT} = 1 + 0.05 \times (5 + 0.10 \times (20 + 0.20 \times 100))$$
$$\text{AMAT} = 1 + 0.05 \times (5 + 0.10 \times 40)$$
$$\text{AMAT} = 1 + 0.05 \times 9$$
$$\text{AMAT} = 1.45 \text{ ns} \quad (\text{vs. } 100 \text{ ns if every access went straight to DRAM} \text{ — } \sim 69\text{x speedup})$$

Adding the Redis tier reduces effective database load: if the cache absorbs 90% of catalog reads at ~0.5 ms and only 10% fall through to a 5 ms DB query, effective read latency = $0.9(0.5) + 0.1(5) = 0.95$ ms, versus 5 ms with no cache — a 5.3x improvement in perceived response time under peak concurrency.

5. Interpretation

The bottleneck at peak load is not CPU cache misses but application-tier cache misses to the database. Introducing a distributed in-memory cache layer, sharding the transactional store, and keeping the hot catalog working set resident in DRAM together cut both latency and disk/database contention, letting the platform sustain far higher concurrent throughput without proportionally scaling database hardware.

Question 2 — Real-Time Robotic Manufacturing System

Scenario: Camera images, sensor readings, and control instructions must be processed under strict, predictable real-time deadlines.

1. Analysis of the Workload

- Control-loop data (PID setpoints, actuator commands): tiny footprint, accessed every cycle, and requires deterministic (bounded worst-case) latency — jitter is as dangerous as average latency.
- Sensor readings (encoders, force sensors, LIDAR): moderate rate, needs low-latency buffering before the control loop consumes it.
- Camera images: large, bursty, and less latency-critical than control data, but still needs to complete within the control cycle window.
- Program/firmware code: mostly read-only during operation but must survive power loss and reboot quickly.

2. Suitability of Memory Technologies

Technology	Suitability for this system	Trade-off
SRAM	Best fit for control-loop variables and tightly-coupled memory (TCM) — sub-ns to few-ns access, no refresh cycles, so latency is fully deterministic	Low density, high cost/bit, high static power — must stay small
DRAM	Good for image frame buffers and larger sensor-fusion working sets that exceed SRAM capacity	Refresh cycles and row-buffer conflicts introduce latency variability — risky for hard-real-time paths unless bank-partitioned

Technology	Suitability for this system	Trade-off
NAND Flash	Suitable for bulk, non-time-critical data logging and firmware images	Slow, block-erase, wear-out — unsuitable for frequently updated control state
MRAM	Excellent for control firmware and critical state: SRAM-like speed with non-volatility, so control parameters survive power loss instantly	Lower density and higher cost than Flash, but ideal where both speed and persistence matter

3. Proposed Memory Hierarchy

- Registers + SRAM scratchpad (Tightly Coupled Memory): the control loop's hot variables live here permanently, bypassing the cache entirely so access time is fixed and predictable — critical for meeting hard deadlines.
- Small, lockable SRAM cache: for sensor-fusion code; cache lines for the control task can be 'locked' so they are never evicted by lower-priority tasks (e.g., image processing), preventing priority inversion in access latency.
- DRAM with a real-time-aware memory controller: used only for image frame buffers, with bank partitioning/priority arbitration so image DMA traffic cannot stall the control core's DRAM accesses.
- MRAM for firmware and live control parameters: instant-on and power-loss safe, avoiding the boot-time latency of loading from Flash.
- NAND Flash for bulk logging: sensor history and diagnostic logs, written asynchronously outside the real-time path.

4. Performance / Predictability Calculation

Worst-case control-loop deadline = 1 ms

SRAM scratchpad access = 2 ns (deterministic, no miss possible)

DRAM (cached) worst case with refresh + row-miss = 150 ns, but only for image DMA, not control path

Because control variables are pinned in SRAM (never miss):

Worst-case control access time = 2 ns -> utilizes < 0.001% of the 1 ms budget

If control data were instead placed in a normal DRAM cache path with a possible cache miss (say 5% miss rate, 150 ns miss penalty):
 $AMAT = 2 + 0.05 \times 150 = 9.5$ ns worst case, still small in absolute terms, but the VARIANCE (jitter) from that 5% miss chance is what breaks hard-real-time guarantees — this is why scratchpad SRAM is preferred over a normal cache here.

5. Interpretation

For hard real-time control, the key metric is not average latency but worst-case bounded latency (determinism). SRAM scratchpad memory and cache locking remove the uncertainty that DRAM refresh and cache eviction would otherwise introduce, while MRAM gives the firmware layer both speed and power-loss resilience that Flash alone cannot provide.

Question 3 — 4K/8K Video-Editing Workstation

Scenario: Multiple concurrent streams of high-resolution 4K/8K video cause processing delays during editing.

1. Analysis of the Workload

- Video frames are large and mostly accessed sequentially (spatial locality within a frame, temporal locality across adjacent frames during scrubbing/playback).
- Working set (multiple streams, timeline previews, effects buffers) far exceeds on-chip cache capacity, so DRAM capacity and bandwidth dominate performance, not cache hit rate alone.
- Storage must sustain very high sustained sequential throughput, not just fast random access.

2. Impact of Each Layer

Layer	Influence on video workload
Cache size (L2/L3)	Helps codec metadata, motion vectors, and small working buffers reused across frames, but cannot hold whole frames (a single 8K 12-bit frame $\approx$ 190 MB raw) — its main job is keeping the decode/encode pipeline's control logic fast.

Layer	Influence on video workload
DRAM capacity	Must hold several seconds of decoded frames per stream plus effect buffers — insufficient capacity forces re-decoding from storage, stalling playback.
DRAM bandwidth	Directly limits how many simultaneous 4K/8K streams can be decoded/composited per second — the primary bottleneck for real-time multi-stream editing.
Storage performance	Determines how fast raw footage streams in from disk; sequential throughput (not IOPS) is what matters most.

3. Proposed Memory Hierarchy

- Per-core L1/L2 caches sized for the codec's working registers and motion-estimation windows.
- Large shared L3 cache to buffer metadata and reduce redundant DRAM fetches across cores working on adjacent frame regions.
- High-bandwidth, multi-channel DRAM (e.g., quad-channel DDR5 or GPU GDDR/HBM for the render pipeline) sized to hold a rolling window of decoded frames for every open stream.
- Hardware/software prefetching that reads frames sequentially ahead of the playhead, exploiting the strong spatial locality of video files.
- NVMe SSDs in RAID-0 (striped) for the active project's media cache — high sequential bandwidth for scrubbing and export.
- Slower HDD/NAS tier for archived raw footage not currently in the timeline.

4. Bandwidth Calculation

8K frame (7680x4320), 12-bit 4:2:2 $\approx 7680 \times 4320 \times 2$ bytes ≈ 66 MB/frame (approx.)

At 30 fps, one 8K stream needs $\approx 66 \text{ MB} \times 30 = 1.98 \text{ GB/s}$

Editing 3 simultaneous 8K streams (e.g., multicam) needs $\approx 3 \times 1.98 \text{ GB/s} \approx 5.9 \text{ GB/s}$

Compare to available bandwidth:

Dual-channel DDR4-3200 DRAM $\approx 25.6 \text{ GB/s}$ -> sufficient for DRAM stage

Single NVMe Gen3 SSD ≈ 3.5 GB/s -> INSUFFICIENT alone for 3 streams

RAID-0 of 2 NVMe SSDs ≈ 7 GB/s -> sufficient with headroom

Conclusion: DRAM is not the bottleneck here; STORAGE bandwidth is — striping

NVMe drives (or caching active footage in DRAM) is the fix.

5. Interpretation

The recommended hierarchy shifts the bottleneck from storage to DRAM by prefetching and striping, and keeps cache usage focused on metadata rather than raw frames. The result is smoother scrubbing, reduced dropped frames, and faster export/render times for multi-stream 4K/8K editing.

Question 4 — Weather Forecasting Numerical Computation

Scenario: Complex numerical simulations run repeated calculations over massive atmospheric datasets (temperature, pressure, wind, humidity).

1. Analysis of the Workload

- Classic HPC/stencil-computation pattern: each grid cell's next value depends on its neighbors, repeated over millions of grid points and thousands of time steps.
- Dataset size (global atmospheric grids) vastly exceeds cache and often exceeds single-node DRAM, so the workload is typically memory-bandwidth-bound, not compute-bound (low arithmetic intensity: few FLOPs per byte fetched).
- Access pattern has strong local spatial locality (neighboring grid cells) but poor reuse without careful blocking, since a naive sweep touches each value once per pass.

2. Influence of Memory Hierarchy

Factor	Effect on performance
Cache hierarchy	Determines how much of a 'tile' of the grid can be reused across neighboring computations before eviction; poor tiling causes repeated DRAM fetches for the same data.

Factor	Effect on performance
DRAM bandwidth	Directly caps the achievable throughput for memory-bound stencil kernels — this is usually the hard performance ceiling (roofline model).
Large-capacity memory	Lets bigger simulation domains or higher grid resolution stay resident, avoiding costly out-of-core disk staging mid-computation.

3. Proposed Memory-Hierarchy Enhancements

- Loop/cache blocking (tiling): restructure the computation to process small sub-grids that fit in L2/L3 cache, maximizing reuse of each loaded value before it is evicted.
- NUMA-aware / multi-channel DRAM allocation: spread the grid across memory channels/sockets so each compute core reads from local, low-contention memory.
- High-bandwidth memory (HBM) or many-channel DDR5 for the compute nodes, since bandwidth — not capacity — is usually the limiting resource.
- Software prefetching for the next tile while the current tile is being computed, hiding DRAM latency behind computation.
- Parallel/striped storage (parallel file system across many SSDs) to stage large datasets into DRAM quickly at the start of each run, and checkpoint results without stalling computation.
- Data compression for less latency-sensitive fields, trading some CPU cycles for reduced DRAM/storage bandwidth pressure.

4. Performance Calculation (Roofline-style estimate)

Kernel arithmetic intensity (I) = 0.5 FLOPs/byte (typical for a simple stencil)
Peak DRAM bandwidth (B) = 100 GB/s
Attainable performance if memory-bound = $I \times B = 0.5 \times 100 = 50$ GFLOP/s

If the CPU's peak compute capability is 400 GFLOP/s, the kernel only reaches $50 / 400 = 12.5\%$ of peak compute — confirming the workload is memory-bound, not compute bound.

With tiling that raises effective reuse (arithmetic intensity) to $I' = 2$ FLOPs/byte:
Attainable performance = $2 \times 100 = 200$ GFLOP/s $\rightarrow$ 4x speedup from blocking alone,
with no change to the CPU or the DRAM hardware.

5. Interpretation

Because the workload is memory-bandwidth bound, the biggest wins come from increasing effective arithmetic intensity (cache blocking) and raw DRAM bandwidth (HBM, more channels, NUMA locality) rather than from a faster CPU. This directly improves data locality, reduces memory-access bottlenecks, and raises sustained computational throughput for the forecasting model.

Question 5 — Smart Home Gateway (Low-Power, Always-On)

Scenario: A gateway continuously aggregates data from cameras, motion sensors, environmental sensors, and appliances, under tight energy constraints and with a need for reliable long-term storage.

1. Analysis of the Workload

- Always-on operation means static (leakage) power matters as much as dynamic power — every idle memory bank still costs energy.
- Data has mixed characteristics: high-frequency small sensor samples (need fast, frequent access), occasional bursty camera frames, and infrequent but must-persist event logs/settings.
- Power loss (unplugging, brownouts) must not corrupt device state — favors non-volatile technologies for critical data.

2. Comparison of Memory Technologies

Technology	Latency	Persistence	Power	Best use here
SRAM	Fastest (~ns)	Volatile	Low dynamic, but leaks continuously	Small working cache for the sensor-polling loop

Technology	Latency	Persistence	Power	Best use here
DRAM (LPDDR)	Fast (~tens of ns)	Volatile	Refresh power even when idle; LPDDR adds self-refresh/sleep modes	Active application buffers, only kept powered when in use
NAND Flash	Slow (μ s–ms), block erase	Non-volatile	Very low idle power, higher write energy, limited write endurance	Bulk long-term storage: recorded clips, historical logs
MRAM / RRAM	Fast (~ns–tens of ns)	Non-volatile	Low idle power, no refresh needed, good write endurance	Frequently updated but must-survive-power-loss data: device state, recent event log, configuration

3. Proposed Memory Hierarchy

- Small SRAM cache for the sensor-polling and communication-stack code — kept minimal to limit static leakage.
- LPDDR DRAM for active buffers (video encode buffer, network stack), placed into self-refresh or powered down entirely between bursts of activity via aggressive duty-cycling.
- MRAM (or RRAM) as a unified 'fast + persistent' tier for frequently changing but critical data — current device state, most-recent sensor readings, and rolling event log — so a power loss never loses recent state and there is no flash-style wear concern from frequent writes.
- NAND Flash for bulk, infrequently written data — archived clips, long-term historical logs, firmware images — where its higher latency and write-endurance limits are acceptable given the low write frequency.
- Power-gating unused memory banks and duty-cycling the camera/DRAM pipeline so peripherals wake only when motion or scheduled sampling triggers activity.

4. Power/Latency Trade-off Illustration

Assume: DRAM self-refresh idle power = 15 mW, MRAM idle (standby) power $\approx$ 0.05 mW
Device is idle 90% of the time, active 10% (typical smart-home duty cycle)

If device state/log is kept in DRAM (must stay refreshed even when idle):

Avg power (state memory only) = $0.9 \times 15 + 0.1 \times 15 = 15$ mW (always paying refresh cost)

If device state/log is moved to MRAM (no refresh needed when idle):

Avg power (state memory only) = $0.9 \times 0.05 + 0.1 \times (\text{active read/write power, say } 5 \text{ mW})$

$$= 0.045 + 0.5 = 0.545 \text{ mW}$$

Roughly a 27x reduction in average power for that portion of memory by choosing MRAM over DRAM for the always-resident state data.

5. Interpretation

For an always-on, energy-constrained gateway, the biggest gain is replacing DRAM's continuous refresh cost with non-volatile MRAM/RRAM wherever data must be both fast and frequently updated, reserving DRAM only for transient buffers that can be power-gated, and pushing bulk cold data to Flash. This hierarchy minimizes standby energy while keeping frequently used data instantly accessible and safe across power loss.